

LAPORAN PRAKTIKUM

STRUKTUR DATA

“Algoritma Sorting”

disusun oleh: Dennis Shauqi Akbar

2511533020

Dosen pengampu: Dr. Wahyudi. S.T.M.T

Asisten Pratikum: Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN

2025

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman. Tugas Individu ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, dengan Judul “Algoritma Sorting”

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pengampu yang telah memberikan arahan sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami.

Padang, 21 Mei 2026

Dennis Shauqi Akbar

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	3
2.1 Pengertian Algoritma Sorting	3
2.2 Implementasi Pada Java.....	3
2.2.1 BubbleSort_2511533020	3
2.2.2 InsertionSort_2511533020.....	4
2.2.3 SelectionSort_2511533020.....	6
2.2.4 InsertionSortGUI_2511533020.....	7
BAB III PENUTUP	10
3.1 Kesimpulan	10
3.2 Saran	10
3.3 Penutup	10
DAFTAR PUSTAKA.....	iii

BAB I

PENDAHULUAN

1.1 Latar Belakang

Sorting atau pengurutan data merupakan salah satu operasi yang paling sering dilakukan dalam pemrograman. Pada praktikum kali ini, saya mempelajari tiga algoritma sorting dasar yaitu Bubble Sort, Insertion Sort, dan Selection Sort. Ketiga algoritma ini termasuk dalam kategori comparison-based sorting karena proses pengurutannya dilakukan dengan membandingkan elemen-elemen data.

Bubble Sort bekerja dengan cara menukar elemen yang berdekatan jika urutannya salah, Insertion Sort bekerja dengan cara menyisipkan elemen ke posisi yang tepat, sedangkan Selection Sort bekerja dengan cara memilih elemen terkecil lalu menukarkannya ke posisi awal. Setiap algoritma memiliki kelebihan dan kekurangan masing-masing tergantung pada kondisi data yang akan diurutkan.

Selain membuat program berbasis console, pada praktikum ini saya juga membuat program GUI (Graphical User Interface) untuk memvisualisasikan proses Insertion Sort langkah demi langkah agar lebih mudah dipahami.

1.2 Tujuan

Tujuan dari praktikum ini adalah memahami cara kerja ketiga algoritma sorting yaitu Bubble Sort, Insertion Sort, dan Selection Sort. Saya diharapkan mampu mengimplementasikan ketiga algoritma tersebut dalam bahasa Java, baik dalam bentuk console program maupun GUI. Selain itu, saya juga belajar membandingkan efisiensi ketiga algoritma berdasarkan jumlah iterasi dan pertukaran yang terjadi.

Pada program GUI Insertion Sort, saya juga belajar bagaimana menampilkan visualisasi proses sorting secara bertahap sehingga alur algoritma dapat diamati dengan jelas.

1.3 Manfaat

Manfaat yang saya peroleh dari praktikum ini adalah pemahaman yang lebih mendalam tentang bagaimana data dapat diurutkan menggunakan berbagai metode. Saya jadi tahu bahwa tidak semua algoritma sorting cocok untuk semua kondisi data. Misalnya Bubble Sort cocok untuk data yang hampir terurut, Insertion Sort bagus untuk data kecil, sedangkan Selection Sort

memiliki jumlah pertukaran yang minimal. Program GUI dibuat juga berguna untuk membantu menjelaskan proses Insertion Sort kepada orang lain karena setiap langkah ditampilkan secara visual. Pengetahuan tentang sorting ini sangat berguna untuk pengembangan aplikasi yang membutuhkan pengurutan data seperti daftar nilai mahasiswa, daftar nama, atau daftar transaksi.

\Selain itu, praktikum ini melatih saya dalam melakukan debugging karena kesalahan pengaturan pointer pada DLL sering menyebabkan program error.

BAB II

PEMBAHASAN

2.1 Pengertian Algoritma Sorting

Sorting adalah proses mengatur sekumpulan data ke dalam urutan tertentu, misalnya dari yang terkecil ke terbesar (ascending) atau dari terbesar ke terkecil (descending). Terdapat banyak algoritma sorting, tiga di antaranya adalah Bubble Sort, Insertion Sort, dan Selection Sort. Bubble Sort bekerja dengan cara membandingkan elemen yang berdekatan dan menukarnya jika tidak sesuai urutan. Proses ini diulang sebanyak $n-1$ kali hingga semua elemen terurut. Insertion Sort bekerja seperti kita mengurutkan kartu di tangan, yaitu mengambil satu elemen lalu menyisipkannya ke posisi yang tepat di antara elemen yang sudah terurut.

Selection Sort bekerja dengan mencari elemen terkecil dari sisa data yang belum terurut, lalu menukarnya ke posisi paling depan. Ketiga algoritma ini memiliki kompleksitas waktu $O(n^2)$ pada kasus terburuk, sehingga lebih cocok untuk data dalam jumlah kecil.

2.2 Implementasi Pada Java

Berikut adalah penjelasan untuk masing-masing kode program Stack yang telah dipraktikkan.

2.2.1 BubbleSort_2511533020

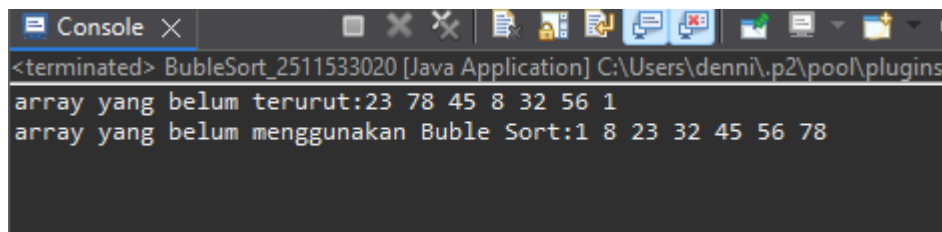
```
1 package pekan7_2511533020;
2
3 public class BubbleSort_2511533020 {
4     public static void bubbleSort (int[] arr) {
5         int n_3020 = arr.length;
6         for (int i_3020 = 0; i_3020 < n_3020; i_3020++) {
7             for (int j_3020 = 0; j_3020 < n_3020 - i_3020 - 1; j_3020++) {
8                 if (arr[j_3020] > arr[j_3020 + 1]) {
9                     int temp_3020 = arr[j_3020];
10                    arr[j_3020] = arr[j_3020 + 1];
11                    arr[j_3020 + 1] = temp_3020;
12                    // System.out.println("data: "+arr[j]+" "+arr[j+1]);
13                }
14            }
15        }
16    }
17    public static void main(String[] args) {
18        int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
19        int n_3020 = arr.length;
20        System.out.print("array yang belum terurut:");
21        for (int i_3020 = 0; i_3020 < n_3020; i_3020++)
22            System.out.print(arr[i_3020] + " ");
23        System.out.println("");
24        bubbleSort(arr);
25        System.out.print("array yang belum menggunakan Buble Sort:");
26        for (int i_3020 = 0; i_3020 < n_3020; i_3020++)
27            System.out.print(arr[i_3020] + " ");
28        System.out.println("");
29    }
30 }
31
32 }
33 }
```

(Gambar 2.2.1)

Program ini mengimplementasikan algoritma Bubble Sort untuk mengurutkan array bilangan bulat. Method bubbleSort menerima parameter array arr dan mengurutkannya secara ascending. Algoritma bekerja dengan dua perulangan bersarang. Perulangan luar (i_3020) mengontrol jumlah pass sebanyak panjang array. Perulangan dalam (j_3020) membandingkan

elemen `arr[j_3020]` dengan `arr[j_3020+1]`. Jika elemen kiri lebih besar dari elemen kanan, maka kedua elemen ditukar menggunakan variabel sementara `temp_3020`. Setelah setiap pass, elemen terbesar akan berpindah ke posisi paling kanan.

Pada method `main`, dibuat array berisi {23, 78, 45, 8, 32, 56, 1}. Array ditampilkan sebelum sorting, kemudian method `bubbleSort` dipanggil, lalu array ditampilkan kembali setelah sorting. Output yang dihasilkan menunjukkan array berubah dari [23, 78, 45, 8, 32, 56, 1] menjadi [1, 8, 23, 32, 45, 56, 78].



2.2.2 InsertionSort_2511533020

```

1 package pekan7_2511533020;
2
3 public class InsertionSort_2511533020 {
4     public static void insertionSort(int[] arr) {
5         int n_3020 = arr.length;
6         for (int i_3020 = 1; i_3020 < n_3020; i_3020++) {
7             int key_3020 = arr[i_3020];
8             int j_3020 = i_3020 - 1;
9             while (j_3020 >= 0 && arr[j_3020] > key_3020) {
10                 arr[j_3020 + 1] = arr[j_3020];
11                 j_3020--;
12             }
13             arr[j_3020 + 1] = key_3020;
14         }
15     }
16 }
17 public static void main(String[] args) {
18     int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
19     int n_3020 = arr.length;
20     System.out.printf("array yang belum terurut:\n");
21     for (int i_3020 = 0; i_3020 < n_3020; i_3020++)
22         System.out.print(arr[i_3020] + " ");
23     System.out.println("");
24     insertionSort(arr);
25     for (int i_3020 = 0; i_3020 < n_3020 ; i_3020++)
26         System.out.print(arr[i_3020] + " ");
27     System.out.println("");
28 }
29
30
31
32
33
34 }
35 }
36

```

(Gambar 2.2.)

Program ini mengimplementasikan algoritma Insertion Sort untuk mengurutkan array bilangan bulat. Method insertionSort bekerja dengan membagi array menjadi dua bagian yaitu bagian yang sudah terurut (kiri) dan bagian yang belum terurut (kanan). Perulangan dimulai dari indeks 1 hingga akhir array. Untuk setiap i_3020 , nilai key_3020 diambil dari $arr[i_3020]$, lalu variabel j_3020 diatur ke i_3020-1 . Selama j_3020 masih ≥ 0 dan $arr[j_3020]$ lebih besar dari key_3020 , maka $arr[j_3020+1]$ digeser ke kanan dan j_3020 dikurangi.

Setelah perulangan selesai, $arr[j_3020+1]$ diisi dengan key_3020 . Proses ini diulang hingga semua elemen terurut. Pada method main, array yang sama dengan Bubble Sort digunakan. Array ditampilkan sebelum dan setelah sorting. Hasil akhirnya sama yaitu array terurut secara ascending.

```
<terminated> InsertionSort_2511533020 [Java Application] C:\Users\denni\.p2\pool\plug
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.2.3 SelectionSort_2511533020

```
1 package pekan7_2511533020;
2
3 public class SelectionSort_2511533020 {
4     public static void selectionSort (int[] arr) {
5         int n_3020 = arr.length;
6         for (int i_3020 = 0; i_3020 < n_3020; i_3020++) {
7             int minIndex_3020 = i_3020;
8             for (int j_3020 = i_3020 + 1; j_3020 < n_3020; j_3020++) {
9                 if (arr[j_3020] < arr[minIndex_3020]) {
10                     minIndex_3020 = j_3020;
11                 }
12             }
13             int temp_3020 = arr[i_3020];
14             arr[i_3020] = arr[minIndex_3020];
15             arr[minIndex_3020] = temp_3020;
16         }
17     }
18
19     public static void main(String[] args) {
20         int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
21         int n_3020 = arr.length;
22         System.out.printf("array yang belum terurut:\n");
23         for (int i_3020 = 0; i_3020 < n_3020; i_3020++)
24             System.out.print(arr[i_3020] + " ");
25         System.out.println("");
26         selectionSort(arr);
27         System.out.printf("array yang terurut:\n");
28         for (int i_3020 = 0; i_3020 < n_3020; i_3020++)
29             System.out.print(arr[i_3020] + " ");
30         System.out.println("");
31     }
32 }
33
34
35 }
36
```

(Gambar 2.2.3)

Program ini mengimplementasikan algoritma Selection Sort. Method selectionSort bekerja dengan mencari nilai terkecil dari sisa array yang belum terurut, lalu menukarnya ke posisi yang tepat. Perulangan luar (i_3020) berjalan dari awal hingga akhir array. Untuk setiap

i_3020 , variabel $minIndex_3020$ diatur ke i_3020 . Perulangan dalam (j_3020) berjalan dari i_3020+1 hingga akhir array. Jika ditemukan nilai $arr[j_3020]$ yang lebih kecil dari $arr[minIndex_3020]$, maka $minIndex_3020$ diupdate. Setelah perulangan dalam selesai, nilai pada indeks i_3020 ditukar dengan nilai pada indeks $minIndex_3020$ menggunakan variabel sementara $temp_3020$.

Dengan cara ini, elemen terkecil akan ditempatkan di posisi awal, kemudian elemen terkecil kedua di posisi kedua, dan seterusnya. Pada method main, array yang sama digunakan dan hasilnya juga sama yaitu array terurut.

```
<terminated> SelectionSort_2511533020 [Java Application] C:\User...
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.2.4 InsertionSortGUI_2511533020

```
1 package pekan7_2511533020;
2
3 import java.awt.BorderLayout;
4
5 public class InsertionSortGUI_2511533020 extends JFrame {
6     private static final long serialVersionUID = 1L;
7     private int[] array_3020;
8     private JLabel[] labelArray_3020;
9     private JButton stepButton_3020, resetButton_3020, setButton_3020;
10    private JTextField inputField_3020;
11    private JPanel panelArray_3020;
12    private JTextArea stepArea_3020;
13
14    private int i_3020 = 1, j_3020;
15    private boolean sorting_3020 = false;
16    private int stepCount_3020 = 1;
17
18    /**
19     * Launch the application.
20     */
21    /**
22     * Create the frame.
23     */
24    public InsertionSortGUI_2511533020() {
25        setTitle("Insertion Sort Langkah per Langkah");
26        setSize(750, 480);
27        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
28        setLocationRelativeTo(null);
29        setLayout(new BorderLayout());
30
31        // Panel input
32        JPanel inputPanel = new JPanel(new FlowLayout());
33        inputField_3020 = new JTextField(30);
34        setButton_3020 = new JButton("Set Array");
35        inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
36        inputPanel.add(inputField_3020);
37        inputPanel.add(setButton_3020);
38
39        panelArray_3020 = new JPanel();
40        panelArray_3020.setLayout(new FlowLayout());
41
42        JPanel controlPanel = new JPanel();
43        stepButton_3020 = new JButton("Langkah Selanjutnya");
44        resetButton_3020 = new JButton("Reset");
45        controlPanel.add(stepButton_3020);
46        controlPanel.add(resetButton_3020);
47
48        stepArea_3020 = new JTextArea(8, 60);
49        stepArea_3020.setEditable(false);
50        stepArea_3020.setFont(new Font("Monospaced", Font.PLAIN, 14));
51        JScrollPane scrollPane = new JScrollPane(stepArea_3020);
52
53        add(inputPanel, BorderLayout.NORTH);
54        add(panelArray_3020, BorderLayout.CENTER);
55        add(controlPanel, BorderLayout.SOUTH);
56        add(scrollPane, BorderLayout.EAST);
57
58        setButton_3020.addActionListener(e -> setArrayFromInput_3020());
59        stepButton_3020.addActionListener(e -> performStep_3020());
60        resetButton_3020.addActionListener(e -> reset_3020());
61
62    }
63
64    private void setArrayFromInput_3020() {
65        String text = inputField_3020.getText().trim();
66        if (text.isEmpty()) return;
67        String[] parts = text.split(",");
68        array_3020 = new int[parts.length];
69        for (int k = 0; k < parts.length; k++) {
70            array_3020[k] = Integer.parseInt(parts[k].trim());
71        }
72        labelArray_3020 = new JLabel[array_3020.length];
73        for (int k = 0; k < array_3020.length; k++) {
74            labelArray_3020[k] = new JLabel(String.valueOf(array_3020[k]));
75        }
76        panelArray_3020.removeAll();
77        for (int k = 0; k < array_3020.length; k++) {
78            panelArray_3020.add(labelArray_3020[k]);
79        }
80        panelArray_3020.revalidate();
81        panelArray_3020.repaint();
82    }
83
84    private void performStep_3020() {
85        if (sorting_3020) return;
86        sorting_3020 = true;
87        stepCount_3020++;
88        if (stepCount_3020 > 10) {
89            sorting_3020 = false;
90            stepCount_3020 = 1;
91        }
92        if (i_3020 < array_3020.length) {
93            j_3020 = i_3020 + 1;
94            while (j_3020 < array_3020.length && array_3020[j_3020] < array_3020[i_3020]) {
95                j_3020++;
96            }
97            if (i_3020 != j_3020) {
98                int temp = array_3020[i_3020];
99                array_3020[i_3020] = array_3020[j_3020];
100                array_3020[j_3020] = temp;
101            }
102            i_3020++;
103            stepArea_3020.setText("Step " + stepCount_3020 + ": array = ");
104            for (int k = 0; k < array_3020.length; k++) {
105                stepArea_3020.append(String.valueOf(array_3020[k]) + " ");
106            }
107            stepArea_3020.append("\n");
108        } else {
109            sorting_3020 = false;
110            stepArea_3020.setText("Array sudah terurut.");
111        }
112    }
113
114    private void reset_3020() {
115        sorting_3020 = false;
116        stepCount_3020 = 1;
117        i_3020 = 1;
118        j_3020 = 0;
119        stepArea_3020.setText("");
120        panelArray_3020.removeAll();
121        panelArray_3020.revalidate();
122        panelArray_3020.repaint();
123    }
124}
```

```

91         array_3020[k] = Integer.parseInt(parts[k].trim());
92     }
93     } catch (NumberFormatException e) {
94         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
95             + "dengan koma", "Error", JOptionPane.ERROR_MESSAGE);
96     }
97     }
98     i_3020 = i;
99     stepCount_3020 = 1;
100     sorting_3020 = true;
101     stepButton_3020.setEnabled(true);
102     stepArea_3020.setText("");
103     panelArray_3020.removeAll();
104     labelArray_3020 = new JLabel[array_3020.length];
105     for (int k = 0; k < array_3020.length; k++) {
106         labelArray_3020[k] = new JLabel(String.valueOf(array_3020[k]));
107         labelArray_3020[k].setFont(new Font("Arial", Font.BOLD, 24));
108         labelArray_3020[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
109         labelArray_3020[k].setPreferredSize(new Dimension(50, 50));
110         labelArray_3020[k].setHorizontalAlignment(SwingConstants.CENTER);
111         panelArray_3020.add(labelArray_3020[k]);
112     }
113     panelArray_3020.revalidate();
114     panelArray_3020.repaint();
115 }
116 private void performStep_3020() {
117     if (i_3020 < array_3020.length && sorting_3020) {
118         int key = array_3020[i_3020];
119         int j_3020 = i_3020 - 1;
120
121         StringBuilder stepLog = new StringBuilder();
122         stepLog.append("Langkah ").append(stepCount_3020).append("\n");
123         stepLog.append("Masukkan ").append(key).append("\n");
124
125         while (j_3020 >= 0 && array_3020[j_3020] > key) {
126             array_3020[j_3020 + 1] = array_3020[j_3020];

```

```

127             j_3020--;
128         }
129         array_3020[j_3020 + 1] = key;
130
131         updateLabels();
132         stepLog.append("Hasil: ").append(arrayToString(array_3020)).append("\n\n");
133         stepArea_3020.append(stepLog.toString());
134
135         i_3020++;
136         stepCount_3020++;
137
138     if (i_3020 == array_3020.length) {
139         sorting_3020 = false;
140         stepButton_3020.setEnabled(false);
141         JOptionPane.showMessageDialog(this, "Sorting selesai!");
142     }
143 }
144 private void updateLabels() {
145     for (int k = 0; k < array_3020.length; k++) {
146         labelArray_3020[k].setText(String.valueOf(array_3020[k]));
147     }
148 }
149 private void reset_3020() {
150     inputField_3020.setText("");
151     panelArray_3020.removeAll();
152     panelArray_3020.revalidate();
153     panelArray_3020.repaint();
154     stepArea_3020.setText("");
155     stepButton_3020.setEnabled(false);
156     sorting_3020 = false;
157     i_3020 = 1;
158     stepCount_3020 = 1;
159 }
160 }
161 }
162 }

```

```

163 private String arrayToString(int[] arr_3020) {
164     StringBuilder sb = new StringBuilder();
165     for (int k = 0; k < arr_3020.length; k++) {
166         sb.append(arr_3020[k]);
167         if (k < arr_3020.length - 1) sb.append(", ");
168     }
169     return sb.toString();
170 }
171 }
172 public static void main(String[] args) {
173     SwingUtilities.invokeLater(() -> {
174         InsertionSortGUI_2511533020 gui = new InsertionSortGUI_2511533020();
175         gui.setVisible(true);
176     });
177 }
178 }

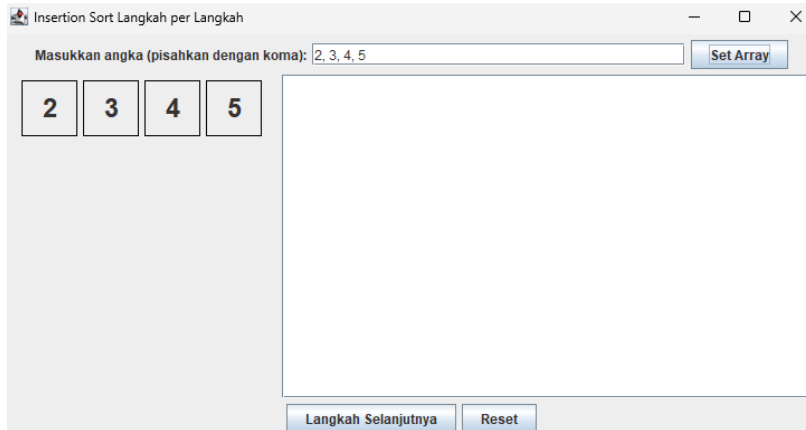
```

(Gambar 2.2.4)

Program ini adalah program GUI (Graphical User Interface) yang memvisualisasikan proses Insertion Sort langkah demi langkah. GUI dibuat menggunakan Java Swing. Program terdiri dari beberapa komponen: JTextField untuk memasukkan angka yang dipisahkan koma, JButton "Set Array" untuk memproses input, JButton "Langkah Selanjutnya" untuk menjalankan sorting satu langkah, JButton "Reset" untuk mengembalikan ke keadaan awal, JPanel untuk menampilkan array dalam bentuk label visual, dan JTextArea untuk menampilkan log setiap langkah. Saat pengguna mengklik "Set Array", input diurai menjadi array integer, lalu ditampilkan dalam bentuk kotak-kotak berisi angka. Tombol "Langkah Selanjutnya" akan

aktif. Setiap kali tombol ditekan, program menjalankan satu iterasi dari algoritma Insertion Sort (memasukkan satu elemen ke posisi yang tepat). Log langkah ditampilkan di JTextArea, dan label-label di panel diperbarui. Proses berlanjut hingga semua elemen terurut, lalu tombol dinonaktifkan dan muncul pesan "Sorting selesai!".

Program ini sangat membantu untuk memahami alur Insertion Sort secara visual



BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa ketiga algoritma sorting (Bubble Sort, Insertion Sort, dan Selection Sort) berhasil diimplementasikan dalam bahasa Java. Ketiganya dapat mengurutkan array bilangan bulat dari nilai terkecil ke terbesar dengan hasil yang sama. Perbedaan utama terletak pada cara kerja dan jumlah perbandingan serta pertukaran yang dilakukan.

Bubble Sort melakukan banyak pertukaran, Insertion Sort efisien untuk data yang hampir terurut, sedangkan Selection Sort memiliki jumlah pertukaran yang minimal. Program GUI Insertion Sort yang dibuat juga berhasil menampilkan visualisasi proses sorting langkah demi langkah sehingga memudahkan pemahaman terhadap alur algoritma

3.2 Saran

Untuk pengembangan lebih lanjut, sebaiknya program Bubble Sort, Insertion Sort, dan Selection Sort dilengkapi dengan penghitung jumlah iterasi dan pertukaran agar dapat membandingkan efisiensi ketiga algoritma secara langsung. Pada program GUI Insertion Sort, sebaiknya ditambahkan fitur untuk memilih kecepatan langkah (delay) atau fitur sort otomatis tanpa harus klik tombol terus menerus.

Selain itu, ketiga algoritma ini dapat dikembangkan untuk mengurutkan data bertipe String (misalnya nama mahasiswa) menggunakan `compareToIgnoreCase()`. Secara umum, kode yang dibuat sudah rapi dan mengikuti aturan penamaan dengan akhiran NIM 3020

3.3 Penutup

Demikian laporan praktikum Algoritma Sorting ini saya buat. Laporan ini berisi penjelasan mengenai tiga algoritma sorting yaitu Bubble Sort, Insertion Sort, dan Selection Sort, serta implementasinya dalam bahasa Java baik versi console maupun GUI.

Saya menyadari bahwa laporan ini masih memiliki kekurangan, oleh karena itu kritik dan saran yang membangun sangat saya harapkan untuk perbaikan di masa mendatang.

DAFTAR PUSTAKA

- [1] Oracle "Arrays (Java Platform SE 17)," Oracle. [Online]. Available: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Arrays.html> (diakses: 21 Mei 2026).
- [2] W3Schools, "Java Sorting Algorithms," W3Schools. [Online]. Available: https://www.w3schools.com/java/java_sorting.asp (diakses: 21 Mei 2026).
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [4] Wikipedia, "Sorting algorithm," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Sorting_algorithm (diakses: 21 Mei 2026).